

Lab 4 Vibe-coding - 58_color_pattern - PES1UG24CS913

Darshana S

Link to chat: <https://claude.ai/share/7f2f8176-c05d-41f4-8168-0397453f8db7>

Assignment: Use Vibe-coding tools to fix and add features to Memory Colour Pattern Game

Tool used: Claude



I'm doing a lab on fixing and extending a pygame Simon-style memory game. I'll give you 4 tasks one at a time. For each, give me the complete updated file, not snippets. Here is the current code:

10/7/2026, 11:00:07 AM



Got it. I've read through all three files (`game_engine.py`, `color_button.py`, and `main.py`). Send over Task 1 whenever you're ready, and I'll return the complete updated file for each one.

While reading, I noticed one thing worth flagging up front, since it may be part of the lab:

`start_next_round` has a bug in the sequence line:

```
self.sequence += self.sequence + [new_color]
```

This doubles the existing sequence every round (`seq += seq + [x]` appends a full copy of `seq` plus the new color), so the pattern grows exponentially instead of by one step. The intended line is `self.sequence.append(new_color)`

I'll hold off on changing anything until you give me the first task, unless it turns out to be exactly this fix. Go ahead when you're ready.

10/7/2026, 11:00:09 AM

Task 1: Fix the sequence duplication bug

Advancing to the next round causes the memory pattern to explode in length rather than extending by one step. Ensure that each successful round strictly appends a single new color step to the active sequence.

10/20/25, 11:02:11 AM



Fixing a bug in the game engine file. >

Only `game_engine.py` changes for this task. The fix is in `start_next_round:` `self.sequence += self.sequence + [new_color]` becomes `self.sequence.append(new_color)`.

```
import random
import pygame
from game.color_button import ColorButton

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        pad_size = 130
        gap = 24
        start_x = width // 2 - pad_size - (gap // 2)
        start_y = 150

        self.buttons = [
            ColorButton(0, pygame.Rect(start_x, start_y, pad_size, pad_size), (110, 20, 20), (255, 50, 50)), # Red
            ColorButton(1, pygame.Rect(start_x + pad_size + gap, start_y, pad_size, pad_size), (15, 60, 150), (40, 170, 255)), # Blue
            ColorButton(2, pygame.Rect(start_x, start_y + pad_size + gap, pad_size, pad_size), (15, 100, 30), (50, 255, 90)), # Green
            ColorButton(3, pygame.Rect(start_x + pad_size + gap, start_y + pad_size + gap, pad_size, pad_size), (140, 110, 10), (255, 235, 40)), #
        ]

        self.sequence = []
        self.player_input = []
        self.score = 0

        self.state = "WATCH"
        self.showing_step = 0
        self.step_start_time = 0
        self.flash_duration = 450
        self.pause_duration = 200
        self.is_flashing = False

        self.player_lit_button = None
        self.player_lit_start = 0
        self.player_flash_duration = 150

        self.font_title = pygame.font.SysFont(None, 40)
        self.font_medium = pygame.font.SysFont(None, 28)

        self.start_next_round()

    def start_next_round(self):
        new_color = random.randint(0, 3)

        # Append exactly one new step to the existing sequence
        self.sequence.append(new_color)

        self.player_input.clear()
        self.state = "WATCH"
        self.showing_step = 0
        self.step_start_time = pygame.time.get_ticks()
        self.is_flashing = True
        self.buttons[self.sequence[0]].is_lit = True

    def update(self):
        now = pygame.time.get_ticks()

        if self.player_lit_button is not None:
            if now - self.player_lit_start >= self.player_flash_duration:
                self.player_lit_button.is_lit = False
                self.player_lit_button = None
```

```

if self.state == "WATCH":
    current_btn_id = self.sequence[self.showing_step]

    if self.is_flashing:
        if now - self.step_start_time >= self.flash_duration:
            self.buttons[current_btn_id].is_lit = False
            self.is_flashing = False
            self.step_start_time = now
        else:
            if now - self.step_start_time >= self.pause_duration:
                self.showing_step += 1
                if self.showing_step < len(self.sequence):
                    next_id = self.sequence[self.showing_step]
                    self.buttons[next_id].is_lit = True
                    self.is_flashing = True
                    self.step_start_time = now
            else:
                self.state = "PLAYER_TURN"

def handle_event(self, event):
    if self.state == "GAME_OVER":
        if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
            self.reset()
        return

    if self.state == "PLAYER_TURN" and event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
        for btn in self.buttons:
            if btn.contains(event.pos):
                btn.is_lit = True
                self.player_lit_button = btn
                self.player_lit_start = pygame.time.get_ticks()

                self.register_player_click(btn.color_id)
                break

def register_player_click(self, color_id):
    self.player_input.append(color_id)
    current_idx = len(self.player_input) - 1

    if self.player_input[current_idx] != self.sequence[current_idx]:
        self.state = "GAME_OVER"
        return

    if len(self.player_input) == len(self.sequence):
        self.score += 1
        self.start_next_round()

def reset(self):
    self.sequence.clear()
    self.player_input.clear()
    self.score = 0
    for btn in self.buttons:
        btn.is_lit = False
    self.player_lit_button = None
    self.start_next_round()

def render(self, screen):
    screen.fill((22, 24, 30))

    title_surf = self.font_title.render("Memory Pattern Arena", True, (245, 245, 245))
    screen.blit(title_surf, (self.width // 2 - title_surf.get_width() // 2, 20))

    score_surf = self.font_medium.render(f"Score: {self.score}", True, (255, 220, 80))
    screen.blit(score_surf, (self.width // 2 - score_surf.get_width() // 2, 60))

    status_text = "Watch the pattern..." if self.state == "WATCH" else "Your turn: Click the pattern!"
    status_color = (190, 195, 205) if self.state == "WATCH" else (80, 240, 130)
    status_surf = self.font_medium.render(status_text, True, status_color)
    screen.blit(status_surf, (self.width // 2 - status_surf.get_width() // 2, 95))

    for btn in self.buttons:
        btn.render(screen)

    if self.state == "GAME_OVER":

```

```

overlay = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
overlay.fill((0, 0, 0, 200))
screen.blit(overlay, (0, 0))

over_surf = self.font_title.render("WRONG PATTERN! GAME OVER", True, (240, 70, 70))
screen.blit(over_surf, (self.width // 2 - over_surf.get_width() // 2, self.height // 2 - 40))

final_score_surf = self.font_medium.render(f"Final Score: {self.score}", True, (255, 255, 255))
screen.blit(final_score_surf, (self.width // 2 - final_score_surf.get_width() // 2, self.height // 2 + 10))

restart_surf = self.font_medium.render("Press [R] to Play Again", True, (200, 200, 200))
screen.blit(restart_surf, (self.width // 2 - restart_surf.get_width() // 2, self.height // 2 + 50))

```

`color_button.py` and `main.py` are unchanged. Round 1 now has 1 step, round 2 has 2, round 3 has 3, and so on. `reset()` clears the sequence before calling `start_next_round()`, so a restart correctly begins at one step again.

Send Task 2 when you're ready.

10/7/2026, 11:02:23 AM

Task 2: Implement dynamic playback acceleration

Sequences flash at the exact same slow, fixed speed throughout every round. Make the playback dynamically accelerate by decreasing the flash and pause durations as the player advances further into higher rounds.

Keep it playable by setting a minimum flash and pause duration, and use named constants. Give me the complete updated `game_engine.py`.

10/7/2026,11:14:06AM



Adding dynamic playback acceleration to the game engine. >

Only `game_engine.py` changes. The base and minimum timings and the per-round speed-up are now named constants at the top of the file. A new `update_playback_speed()` method recalculates `flash_duration` and `pause_duration` at the start of each round, and `max()` keeps them from dropping below the minimums.

```
import random
import pygame
from game.color_button import ColorButton

# Playback timing (milliseconds)
BASE_FLASH_DURATION = 450
BASE_PAUSE_DURATION = 200
MIN_FLASH_DURATION = 150
MIN_PAUSE_DURATION = 80

# How much faster playback gets for each successful round
FLASH_SPEEDUP_PER_ROUND = 25
PAUSE_SPEEDUP_PER_ROUND = 10

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        pad_size = 130
        gap = 24
        start_x = width // 2 - pad_size - (gap // 2)
        start_y = 150

        self.buttons = [
            ColorButton(0, pygame.Rect(start_x, start_y, pad_size, pad_size), (110, 20, 20), (255, 50, 50)), # Red
            ColorButton(1, pygame.Rect(start_x + pad_size + gap, start_y, pad_size, pad_size), (15, 60, 150), (40, 170, 255)), # Blue
            ColorButton(2, pygame.Rect(start_x, start_y + pad_size + gap, pad_size, pad_size), (15, 100, 30), (50, 255, 90)), # Green
            ColorButton(3, pygame.Rect(start_x + pad_size + gap, start_y + pad_size + gap, pad_size, pad_size), (140, 110, 10), (255, 235, 40)), # Yellow
        ]

        self.sequence = []
        self.player_input = []
        self.score = 0

        self.state = "WATCH"
        self.showing_step = 0
        self.step_start_time = 0
        self.flash_duration = BASE_FLASH_DURATION
        self.pause_duration = BASE_PAUSE_DURATION
        self.is_flashing = False

        self.player_lit_button = None
        self.player_lit_start = 0
        self.player_flash_duration = 150

        self.font_title = pygame.font.SysFont(None, 40)
        self.font_medium = pygame.font.SysFont(None, 28)

        self.start_next_round()

    def update_playback_speed(self):
        """Shorten flash and pause durations as the score grows, down to a floor."""
        self.flash_duration = max(
            MIN_FLASH_DURATION,
            BASE_FLASH_DURATION - self.score * FLASH_SPEEDUP_PER_ROUND,
        )
        self.pause_duration = max(
            MIN_PAUSE_DURATION,
```

```

        BASE_PAUSE_DURATION - self.score * PAUSE_SPEEDUP_PER_ROUND,
    )

def start_next_round(self):
    new_color = random.randint(0, 3)

    # Append exactly one new step to the existing sequence
    self.sequence.append(new_color)

    self.update_playback_speed()

    self.player_input.clear()
    self.state = "WATCH"
    self.showing_step = 0
    self.step_start_time = pygame.time.get_ticks()
    self.is_flashing = True
    self.buttons[self.sequence[0]].is_lit = True

def update(self):
    now = pygame.time.get_ticks()

    if self.player_lit_button is not None:
        if now - self.player_lit_start >= self.player_flash_duration:
            self.player_lit_button.is_lit = False
            self.player_lit_button = None

    if self.state == "WATCH":
        current_btn_id = self.sequence[self.showing_step]

        if self.is_flashing:
            if now - self.step_start_time >= self.flash_duration:
                self.buttons[current_btn_id].is_lit = False
                self.is_flashing = False
                self.step_start_time = now
            else:
                if now - self.step_start_time >= self.pause_duration:
                    self.showing_step += 1
                    if self.showing_step < len(self.sequence):
                        next_id = self.sequence[self.showing_step]
                        self.buttons[next_id].is_lit = True
                        self.is_flashing = True
                        self.step_start_time = now
                    else:
                        self.state = "PLAYER_TURN"

def handle_event(self, event):
    if self.state == "GAME_OVER":
        if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
            self.reset()
        return

    if self.state == "PLAYER_TURN" and event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
        for btn in self.buttons:
            if btn.contains(event.pos):
                btn.is_lit = True
                self.player_lit_button = btn
                self.player_lit_start = pygame.time.get_ticks()

                self.register_player_click(btn.color_id)
                break

def register_player_click(self, color_id):
    self.player_input.append(color_id)
    current_idx = len(self.player_input) - 1

    if self.player_input[current_idx] != self.sequence[current_idx]:
        self.state = "GAME_OVER"
        return

    if len(self.player_input) == len(self.sequence):
        self.score += 1
        self.start_next_round()

def reset(self):
    self.sequence.clear()

```

```

self.player_input.clear()
self.score = 0
for btn in self.buttons:
    btn.is_lit = False
self.player_lit_button = None
self.start_next_round()

def render(self, screen):
    screen.fill((22, 24, 30))

    title_surf = self.font_title.render("Memory Pattern Arena", True, (245, 245, 245))
    screen.blit(title_surf, (self.width // 2 - title_surf.get_width() // 2, 20))

    score_surf = self.font_medium.render(f"Score: {self.score}", True, (255, 220, 80))
    screen.blit(score_surf, (self.width // 2 - score_surf.get_width() // 2, 60))

    status_text = "Watch the pattern..." if self.state == "WATCH" else "Your turn: Click the pattern!"
    status_color = (190, 195, 205) if self.state == "WATCH" else (80, 240, 130)
    status_surf = self.font_medium.render(status_text, True, status_color)
    screen.blit(status_surf, (self.width // 2 - status_surf.get_width() // 2, 95))

    for btn in self.buttons:
        btn.render(screen)

    if self.state == "GAME_OVER":
        overlay = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
        overlay.fill((0, 0, 0, 200))
        screen.blit(overlay, (0, 0))

        over_surf = self.font_title.render("WRONG PATTERN! GAME OVER", True, (240, 70, 70))
        screen.blit(over_surf, (self.width // 2 - over_surf.get_width() // 2, self.height // 2 - 40))

        final_score_surf = self.font_medium.render(f"Final Score: {self.score}", True, (255, 255, 255))
        screen.blit(final_score_surf, (self.width // 2 - final_score_surf.get_width() // 2, self.height // 2 + 10))

        restart_surf = self.font_medium.render("Press [R] to Play Again", True, (200, 200, 200))
        screen.blit(restart_surf, (self.width // 2 - restart_surf.get_width() // 2, self.height // 2 + 50))

```

- **Speed curve:** Round 1 (score 0) plays at the original 450 ms flash and 200 ms pause. Each successful round trims 25 ms from the flash and 10 ms from the pause. Both hit their floors (150 ms and 80 ms) at score 12 and stay there.
- **Restart:** `reset()` sets the score to 0 before calling `start_next_round()`, so a new game starts back at the slow speed.
- **Tuning:** To change the difficulty curve, edit the constants at the top of the file.
- **Unchanged:** `color_button.py` and `main.py` are untouched. `player_flash_duration` (the brief highlight when the player clicks) is also unchanged, since it isn't part of playback.

Send Task 3 when you're ready.

10/7/2026, 11:42:22 AM

Task 3: Implement sound effects or audio frequencies

The memory puzzle is completely silent, relying only on visual illumination. Assign a distinct musical pitch or audio frequency to each of the four colored pads that sounds whenever a tile is flashed or clicked.

Generate the tones in code so I don't need audio files. If no audio device is available, the game should still run. Give me the complete updated

`game_engine.py`

10/7/2026, 11:18:57 AM



Looping tones smoothly and handling edge-case timing quirks. >

Only `game_engine.py` changes. Tones are generated at startup as sine waves using only the standard library (`math` and `array`), so there's no numpy dependency and no audio files. Each pad's tone loops while its pad is lit and fades out when the light goes off, so the sound length follows the flash length, even as playback speeds up. If the mixer can't be opened, `sound_enabled` stays `False` and the game runs silently.

```
import math
import random
from array import array

import pygame
from game.color_button import ColorButton

# Playback timing (milliseconds)
BASE_FLASH_DURATION = 450
BASE_PAUSE_DURATION = 200
MIN_FLASH_DURATION = 150
MIN_PAUSE_DURATION = 80

# How much faster playback gets for each successful round
FLASH_SPEEDUP_PER_ROUND = 25
PAUSE_SPEEDUP_PER_ROUND = 10

# Audio settings
SAMPLE_RATE = 44100
TONE_VOLUME = 0.3          # 0.0 - 1.0 of full scale
TONE_LOOP_SECONDS = 0.5    # approximate length of the looped tone buffer
TONE_FADE_IN_MS = 10       # softens the start of a tone to avoid clicks
TONE_FADE_OUT_MS = 40      # softens the end of a tone to avoid clicks

# One pitch per pad, in Hz: a C major arpeggio (C4, E4, G4, C5)
PAD_FREQUENCIES = [
    261.63, # Red
    329.63, # Blue
    392.00, # Green
    523.25, # Yellow
]

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        pad_size = 130
        gap = 24
        start_x = width // 2 - pad_size - (gap // 2)
        start_y = 150

        self.buttons = [
            ColorButton(0, pygame.Rect(start_x, start_y, pad_size, pad_size), (110, 20, 20), (255, 50, 50)), # Red
            ColorButton(1, pygame.Rect(start_x + pad_size + gap, start_y, pad_size, pad_size), (15, 60, 150), (40, 170, 255)), # Blue
            ColorButton(2, pygame.Rect(start_x, start_y + pad_size + gap, pad_size, pad_size), (15, 100, 30), (50, 255, 90)), # Green
            ColorButton(3, pygame.Rect(start_x + pad_size + gap, start_y + pad_size + gap, pad_size, pad_size), (140, 110, 10), (255, 235, 40)), # Yellow
        ]

        self.sequence = []
        self.player_input = []
        self.score = 0

        self.state = "WATCH"
        self.showing_step = 0
        self.step_start_time = 0
        self.flash_duration = BASE_FLASH_DURATION
```

```

self.pause_duration = BASE_PAUSE_DURATION
self.is_flashing = False

self.player_lit_button = None
self.player_lit_start = 0
self.player_flash_duration = 150

self.font_title = pygame.font.SysFont(None, 40)
self.font_medium = pygame.font.SysFont(None, 28)

self.sound_enabled = False
self.tones = []
self.init_audio()

self.start_next_round()

# -----
# Audio
# -----
def init_audio(self):
    """Set up the mixer and build one tone per pad. Fails silently."""
    try:
        # pygame.init() may have already opened the mixer with other
        # settings (or failed to open it), so (re) open it ourselves.
        if pygame.mixer.get_init():
            pygame.mixer.quit()
        pygame.mixer.init(frequency=SAMPLE_RATE, size=-16, channels=1)

        settings = pygame.mixer.get_init()
        if settings is None:
            return
        rate, fmt, channels = settings

        # The tone generator writes signed 16-bit samples only.
        if fmt != -16:
            pygame.mixer.quit()
            return

        self.tones = [self.build_tone(freq, rate, channels) for freq in PAD_FREQUENCIES]
        self.sound_enabled = True
    except (pygame.error, NotImplementedError, OSError):
        # No audio device or driver: run the game without sound.
        self.sound_enabled = False
        self.tones = []

def build_tone(self, frequency, rate, channels):
    """Create a loopable sine-wave Sound for the given frequency."""
    # Use a whole number of cycles so the loop point is seamless.
    cycles = max(1, round(frequency * TONE_LOOP_SECONDS))
    num_samples = round(cycles * rate / frequency)
    amplitude = int(32767 * TONE_VOLUME)

    samples = array("h")
    for i in range(num_samples):
        value = int(amplitude * math.sin(2 * math.pi * cycles * i / num_samples))
        for _ in range(channels):
            samples.append(value)

    return pygame.mixer.Sound(buffer=samples.tobytes())

def start_tone(self, color_id):
    if not self.sound_enabled:
        return
    try:
        self.tones[color_id].play(loops=-1, fade_ms=TONE_FADE_IN_MS)
    except pygame.error:
        self.sound_enabled = False

def stop_tone(self, color_id):
    if not self.sound_enabled:
        return
    try:
        self.tones[color_id].fadeout(TONE_FADE_OUT_MS)
    except pygame.error:
        self.sound_enabled = False

```

```

def stop_all_tones(self):
    for color_id in range(len(self.tones)):
        self.stop_tone(color_id)

# -----
# Game logic
# -----

def update_playback_speed(self):
    """Shorten flash and pause durations as the score grows, down to a floor."""
    self.flash_duration = max(
        MIN_FLASH_DURATION,
        BASE_FLASH_DURATION - self.score * FLASH_SPEEDUP_PER_ROUND,
    )
    self.pause_duration = max(
        MIN_PAUSE_DURATION,
        BASE_PAUSE_DURATION - self.score * PAUSE_SPEEDUP_PER_ROUND,
    )

def start_next_round(self):
    new_color = random.randint(0, 3)

    # Append exactly one new step to the existing sequence
    self.sequence.append(new_color)

    self.update_playback_speed()

    self.player_input.clear()
    self.state = "WATCH"
    self.showing_step = 0
    self.step_start_time = pygame.time.get_ticks()
    self.is_flashing = True
    self.buttons[self.sequence[0]].is_lit = True
    self.start_tone(self.sequence[0])

def update(self):
    now = pygame.time.get_ticks()

    if self.player_lit_button is not None:
        if now - self.player_lit_start >= self.player_flash_duration:
            self.player_lit_button.is_lit = False
            self.stop_tone(self.player_lit_button.color_id)
            self.player_lit_button = None

    if self.state == "WATCH":
        current_btn_id = self.sequence[self.showing_step]

        if self.is_flashing:
            if now - self.step_start_time >= self.flash_duration:
                self.buttons[current_btn_id].is_lit = False
                self.stop_tone(current_btn_id)
                self.is_flashing = False
                self.step_start_time = now
            else:
                if now - self.step_start_time >= self.pause_duration:
                    self.showing_step += 1
                    if self.showing_step < len(self.sequence):
                        next_id = self.sequence[self.showing_step]
                        self.buttons[next_id].is_lit = True
                        self.start_tone(next_id)
                        self.is_flashing = True
                        self.step_start_time = now
                    else:
                        self.state = "PLAYER_TURN"

def handle_event(self, event):
    if self.state == "GAME_OVER":
        if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
            self.reset()
        return

    if self.state == "PLAYER_TURN" and event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
        for btn in self.buttons:
            if btn.contains(event.pos):
                btn.is_lit = True

```

```

        self.start_tone(btn.color_id)
        self.player_lit_button = btn
        self.player_lit_start = pygame.time.get_ticks()

        self.register_player_click(btn.color_id)
        break

def register_player_click(self, color_id):
    self.player_input.append(color_id)
    current_idx = len(self.player_input) - 1

    if self.player_input[current_idx] != self.sequence[current_idx]:
        self.state = "GAME_OVER"
        return

    if len(self.player_input) == len(self.sequence):
        self.score += 1
        self.start_next_round()

def reset(self):
    self.stop_all_tones()
    self.sequence.clear()
    self.player_input.clear()
    self.score = 0
    for btn in self.buttons:
        btn.is_lit = False
    self.player_lit_button = None
    self.start_next_round()

def render(self, screen):
    screen.fill((22, 24, 30))

    title_surf = self.font_title.render("Memory Pattern Arena", True, (245, 245, 245))
    screen.blit(title_surf, (self.width // 2 - title_surf.get_width() // 2, 20))

    score_surf = self.font_medium.render(f"Score: {self.score}", True, (255, 220, 80))
    screen.blit(score_surf, (self.width // 2 - score_surf.get_width() // 2, 60))

    status_text = "Watch the pattern..." if self.state == "WATCH" else "Your turn: Click the pattern!"
    status_color = (190, 195, 205) if self.state == "WATCH" else (80, 240, 130)
    status_surf = self.font_medium.render(status_text, True, status_color)
    screen.blit(status_surf, (self.width // 2 - status_surf.get_width() // 2, 95))

    for btn in self.buttons:
        btn.render(screen)

    if self.state == "GAME_OVER":
        overlay = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
        overlay.fill((0, 0, 0, 200))
        screen.blit(overlay, (0, 0))

        over_surf = self.font_title.render("WRONG PATTERN! GAME OVER", True, (240, 70, 70))
        screen.blit(over_surf, (self.width // 2 - over_surf.get_width() // 2, self.height // 2 - 40))

        final_score_surf = self.font_medium.render(f"Final Score: {self.score}", True, (255, 255, 255))
        screen.blit(final_score_surf, (self.width // 2 - final_score_surf.get_width() // 2, self.height // 2 + 10))

        restart_surf = self.font_medium.render("Press [R] to Play Again", True, (200, 200, 200))
        screen.blit(restart_surf, (self.width // 2 - restart_surf.get_width() // 2, self.height // 2 + 50))

```

- **Pitches:** Red is C4 (261.63 Hz), Blue is E4 (329.63 Hz), Green is G4 (392.00 Hz) and Yellow is C5 (523.25 Hz). They form a C major arpeggio, so any sequence sounds consonant. Edit `PAD_FREQUENCIES` to change them.
- **When tones play:** Tones start on every playback flash and every player click, and stop when the pad goes dark, so they track the accelerating flash timing from Task 2.
- **Loop and fades:** Each tone buffer holds a whole number of sine cycles so it loops without a click, and the short fade-in and fade-out remove pops at the start and end.
- **No audio device:** `init_audio()` catches `pygame.error` (and similar failures) and leaves `sound_enabled = False`. Every `start_tone` and `stop_tone` call returns immediately in that case, so nothing else changes. If a playback error happens mid-game, sound is switched off the same way.
- **Mixer reinit:** `pygame.init()` in `main.py` opens the mixer with its own defaults (usually stereo), so `init_audio()` reopens it with a known format and builds the samples to match whatever channel count the device actually gives.
- **Restart:** `reset()` calls `stop_all_tones()` first, so pressing R mid-flash won't leave a note ringing.
- **Unchanged:** `color_button.py` and `main.py` are untouched.

Send Task 4 when you're ready.

10/7/2026, 11:35 AM

Task 4: Implement a Player Turn Input Timer

Players currently have unlimited time to study the board between clicks. Introduce a dynamic countdown timer bar for the player's turn that triggers a game over if time expires before completing the pattern.

Scale the time limit with the length of the sequence, and show "TIME'S UP!" instead of "WRONG PATTERN!" on timeout. Pressing R should still restart. Give me the complete updated `game_engine.py`

10/2026, 11:26:11 AM



Adding a turn timer to the game engine. >

Only `game_engine.py` changes. The player's turn now has a time limit of `TURN_TIME_BASE_MS + TURN_TIME_PER_STEP_MS * len(sequence)` and the clock starts the moment playback ends. A bar under the pads drains as time runs out, shifting from green to yellow to red. If the bar empties before the pattern is complete, the game ends with "TIME'S UP!". A wrong click still shows "WRONG PATTERN! GAME OVER".

```
import math
import random
from array import array

import pygame
from game.color_button import ColorButton

# Playback timing (milliseconds)
BASE_FLASH_DURATION = 450
BASE_PAUSE_DURATION = 200
MIN_FLASH_DURATION = 150
MIN_PAUSE_DURATION = 80

# How much faster playback gets for each successful round
FLASH_SPEEDUP_PER_ROUND = 25
PAUSE_SPEEDUP_PER_ROUND = 10

# Player turn timer (milliseconds)
# Time limit = BASE + PER_STEP * (number of steps in the sequence)
TURN_TIME_BASE_MS = 2000
TURN_TIME_PER_STEP_MS = 800

# Timer bar appearance
TIMER_BAR_WIDTH = 300
TIMER_BAR_HEIGHT = 16
TIMER_BAR_Y = 460
TIMER_COLOR_OK = (50, 255, 90)
TIMER_COLOR_WARN = (255, 235, 40)
TIMER_COLOR_DANGER = (255, 50, 50)
TIMER_WARN_FRACTION = 0.5 # below this fraction of time left, bar turns yellow
TIMER_DANGER_FRACTION = 0.25 # below this fraction of time left, bar turns red

# Reasons the game can end
REASON_WRONG = "WRONG"
REASON_TIMEOUT = "TIMEOUT"

# Audio settings
SAMPLE_RATE = 44100
TONE_VOLUME = 0.3 # 0.0 - 1.0 of full scale
TONE_LOOP_SECONDS = 0.5 # approximate length of the looped tone buffer
TONE_FADE_IN_MS = 10 # softens the start of a tone to avoid clicks
TONE_FADE_OUT_MS = 40 # softens the end of a tone to avoid clicks

# One pitch per pad, in Hz: a C major arpeggio (C4, E4, G4, C5)
PAD_FREQUENCIES = [
    261.63, # Red
    329.63, # Blue
    392.00, # Green
    523.25, # Yellow
]

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        pad_size = 130
        gap = 24
```

Yellow

```
start_x = width // 2 - pad_size - (gap // 2)
start_y = 150

self.buttons = [
    ColorButton(0, pygame.Rect(start_x, start_y, pad_size, pad_size), (110, 20, 20), (255, 50, 50)), # Red
    ColorButton(1, pygame.Rect(start_x + pad_size + gap, start_y, pad_size, pad_size), (15, 60, 150), (40, 170, 255)), # Blue
    ColorButton(2, pygame.Rect(start_x, start_y + pad_size + gap, pad_size, pad_size), (15, 100, 30), (50, 255, 90)), # Green
    ColorButton(3, pygame.Rect(start_x + pad_size + gap, start_y + pad_size + gap, pad_size, pad_size), (140, 110, 10), (255, 235, 40)), #
]

self.sequence = []
self.player_input = []
self.score = 0

self.state = "WATCH"
self.showing_step = 0
self.step_start_time = 0
self.flash_duration = BASE_FLASH_DURATION
self.pause_duration = BASE_PAUSE_DURATION
self.is_flashing = False

self.player_lit_button = None
self.player_lit_start = 0
self.player_flash_duration = 150

# Player turn timer
self.turn_start_time = 0
self.turn_time_limit = TURN_TIME_BASE_MS
self.game_over_reason = REASON_WRONG

self.font_title = pygame.font.SysFont(None, 40)
self.font_medium = pygame.font.SysFont(None, 28)

self.sound_enabled = False
self.tones = []
self.init_audio()

self.start_next_round()

# -----
# Audio
# -----
def init_audio(self):
    """Set up the mixer and build one tone per pad. Fails silently."""
    try:
        # pygame.init() may have already opened the mixer with other
        # settings (or failed to open it), so (re) open it ourselves.
        if pygame.mixer.get_init():
            pygame.mixer.quit()
        pygame.mixer.init(frequency=SAMPLE_RATE, size=-16, channels=1)

        settings = pygame.mixer.get_init()
        if settings is None:
            return
        rate, fmt, channels = settings

        # The tone generator writes signed 16-bit samples only.
        if fmt != -16:
            pygame.mixer.quit()
            return

        self.tones = [self.build_tone(freq, rate, channels) for freq in PAD_FREQUENCIES]
        self.sound_enabled = True
    except (pygame.error, NotImplementedError, OSError):
        # No audio device or driver: run the game without sound.
        self.sound_enabled = False
        self.tones = []

def build_tone(self, frequency, rate, channels):
    """Create a loopable sine-wave sound for the given frequency."""
    # Use a whole number of cycles so the loop point is seamless.
    cycles = max(1, round(frequency * TONE_LOOP_SECONDS))
    num_samples = round(cycles * rate / frequency)
    amplitude = int(32767 * TONE_VOLUME)
```

```

        samples = array("h")
        for i in range(num_samples):
            value = int(amplitude * math.sin(2 * math.pi * cycles * i / num_samples))
            for _ in range(channels):
                samples.append(value)

        return pygame.mixer.Sound(buffer=samples.tobytes())

def start_tone(self, color_id):
    if not self.sound_enabled:
        return
    try:
        self.tones[color_id].play(loops=-1, fade_ms=TONE_FADE_IN_MS)
    except pygame.error:
        self.sound_enabled = False

def stop_tone(self, color_id):
    if not self.sound_enabled:
        return
    try:
        self.tones[color_id].fadeout(TONE_FADE_OUT_MS)
    except pygame.error:
        self.sound_enabled = False

def stop_all_tones(self):
    for color_id in range(len(self.tones)):
        self.stop_tone(color_id)

# -----
# Game logic
# -----

def update_playback_speed(self):
    """Shorten flash and pause durations as the score grows, down to a floor."""
    self.flash_duration = max(
        MIN_FLASH_DURATION,
        BASE_FLASH_DURATION - self.score * FLASH_SPEEDUP_PER_ROUND,
    )
    self.pause_duration = max(
        MIN_PAUSE_DURATION,
        BASE_PAUSE_DURATION - self.score * PAUSE_SPEEDUP_PER_ROUND,
    )

def start_player_turn(self, now):
    """Switch to the player's turn and start the countdown."""
    self.state = "PLAYER_TURN"
    self.turn_start_time = now
    self.turn_time_limit = TURN_TIME_BASE_MS + TURN_TIME_PER_STEP_MS * len(self.sequence)

def get_time_left_fraction(self):
    """Fraction of the turn time still remaining, from 1.0 down to 0.0."""
    elapsed = pygame.time.get_ticks() - self.turn_start_time
    remaining = self.turn_time_limit - elapsed
    return max(0.0, min(1.0, remaining / self.turn_time_limit))

def start_next_round(self):
    new_color = random.randint(0, 3)

    # Append exactly one new step to the existing sequence
    self.sequence.append(new_color)

    self.update_playback_speed()

    self.player_input.clear()
    self.state = "WATCH"
    self.showing_step = 0
    self.step_start_time = pygame.time.get_ticks()
    self.is_flashing = True
    self.buttons[self.sequence[0]].is_lit = True
    self.start_tone(self.sequence[0])

def update(self):
    now = pygame.time.get_ticks()

    if self.player_lit_button is not None:

```

```

        if now - self.player_lit_start >= self.player_flash_duration:
            self.player_lit_button.is_lit = False
            self.stop_tone(self.player_lit_button.color_id)
            self.player_lit_button = None

    if self.state == "WATCH":
        current_btn_id = self.sequence[self.showing_step]

        if self.is_flashing:
            if now - self.step_start_time >= self.flash_duration:
                self.buttons[current_btn_id].is_lit = False
                self.stop_tone(current_btn_id)
                self.is_flashing = False
                self.step_start_time = now
            else:
                if now - self.step_start_time >= self.pause_duration:
                    self.showing_step += 1
                    if self.showing_step < len(self.sequence):
                        next_id = self.sequence[self.showing_step]
                        self.buttons[next_id].is_lit = True
                        self.start_tone(next_id)
                        self.is_flashing = True
                        self.step_start_time = now
                    else:
                        self.start_player_turn(now)

        elif self.state == "PLAYER_TURN":
            if now - self.turn_start_time >= self.turn_time_limit:
                self.game_over_reason = REASON_TIMEOUT
                self.state = "GAME_OVER"

def handle_event(self, event):
    if self.state == "GAME_OVER":
        if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
            self.reset()
        return

    if self.state == "PLAYER_TURN" and event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
        for btn in self.buttons:
            if btn.contains(event.pos):
                btn.is_lit = True
                self.start_tone(btn.color_id)
                self.player_lit_button = btn
                self.player_lit_start = pygame.time.get_ticks()

                self.register_player_click(btn.color_id)
                break

def register_player_click(self, color_id):
    self.player_input.append(color_id)
    current_idx = len(self.player_input) - 1

    if self.player_input[current_idx] != self.sequence[current_idx]:
        self.game_over_reason = REASON_WRONG
        self.state = "GAME_OVER"
        return

    if len(self.player_input) == len(self.sequence):
        self.score += 1
        self.start_next_round()

def reset(self):
    self.stop_all_tones()
    self.sequence.clear()
    self.player_input.clear()
    self.score = 0
    self.game_over_reason = REASON_WRONG
    for btn in self.buttons:
        btn.is_lit = False
    self.player_lit_button = None
    self.start_next_round()

def render_timer_bar(self, screen):
    """Draw the countdown bar under the pads during the player's turn."""
    fraction = self.get_time_left_fraction()

```

```

if fraction > TIMER_WARN_FRACTION:
    color = TIMER_COLOR_OK
elif fraction > TIMER_DANGER_FRACTION:
    color = TIMER_COLOR_WARN
else:
    color = TIMER_COLOR_DANGER

bar_x = self.width // 2 - TIMER_BAR_WIDTH // 2
track_rect = pygame.Rect(bar_x, TIMER_BAR_Y, TIMER_BAR_WIDTH, TIMER_BAR_HEIGHT)
pygame.draw.rect(screen, (45, 48, 58), track_rect, border_radius=8)

fill_width = int(TIMER_BAR_WIDTH * fraction)
if fill_width > 0:
    fill_rect = pygame.Rect(bar_x, TIMER_BAR_Y, fill_width, TIMER_BAR_HEIGHT)
    pygame.draw.rect(screen, color, fill_rect, border_radius=8)

pygame.draw.rect(screen, (70, 75, 85), track_rect, width=2, border_radius=8)

def render(self, screen):
    screen.fill((22, 24, 30))

    title_surf = self.font_title.render("Memory Pattern Arena", True, (245, 245, 245))
    screen.blit(title_surf, (self.width // 2 - title_surf.get_width() // 2, 20))

    score_surf = self.font_medium.render(f"Score: {self.score}", True, (255, 220, 80))
    screen.blit(score_surf, (self.width // 2 - score_surf.get_width() // 2, 60))

    status_text = "Watch the pattern..." if self.state == "WATCH" else "Your turn: Click the pattern!"
    status_color = (190, 195, 205) if self.state == "WATCH" else (80, 240, 130)
    status_surf = self.font_medium.render(status_text, True, status_color)
    screen.blit(status_surf, (self.width // 2 - status_surf.get_width() // 2, 95))

    for btn in self.buttons:
        btn.render(screen)

    if self.state == "PLAYER_TURN":
        self.render_timer_bar(screen)

    if self.state == "GAME_OVER":
        overlay = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
        overlay.fill((0, 0, 0, 200))
        screen.blit(overlay, (0, 0))

        if self.game_over_reason == REASON_TIMEOUT:
            over_text = "TIME'S UP! "
        else:
            over_text = "WRONG PATTERN! GAME OVER"

        over_surf = self.font_title.render(over_text, True, (240, 70, 70))
        screen.blit(over_surf, (self.width // 2 - over_surf.get_width() // 2, self.height // 2 - 40))

        final_score_surf = self.font_medium.render(f"Final Score: {self.score}", True, (255, 255, 255))
        screen.blit(final_score_surf, (self.width // 2 - final_score_surf.get_width() // 2, self.height // 2 + 10))

        restart_surf = self.font_medium.render("Press [R] to Play Again", True, (200, 200, 200))
        screen.blit(restart_surf, (self.width // 2 - restart_surf.get_width() // 2, self.height // 2 + 50))

```

- **Time limit:** The limit is 2000 ms plus 800 ms per step. Round 1 (1 step) gives 2.8 s, a 5-step sequence gives 6 s, and a 10-step sequence gives 10 s. Edit `TURN_TIME_BASE_MS` and `TURN_TIME_PER_STEP_MS` to tune it.
- **Timer start:** `start_player_turn()` records the start time when playback finishes, so the player never loses time while watching the pattern.
- **Timeout check:** `update()` checks the clock every frame during `PLAYER_TURN`. On expiry it sets `game_over_reason` to `REASON_TIMEOUT` and moves to `GAME_OVER`, which also stops the timer bar from being drawn.
- **Game over text:** `render()` picks "TIME'S UP!" or "WRONG PATTERN! GAME OVER" from `game_over_reason`. A wrong click sets the reason to `REASON_WRONG` in `register_player_click()`.
- **Restart:** `reset()` sets the reason back to the default and calls `start_next_round()`, which starts the timer again after playback. Pressing R works after either kind of game over.
- **Bar colors:** The bar is green above 50% of the time left, yellow above 25%, and red below that. The thresholds are constants at the top of the file.
- **Layout:** The bar sits at y=460, just under the pads (which end at y=434) and inside the 520 px window.
- **Unchanged:** `color_button.py` and `main.py` are untouched.

That's all four tasks. If your lab needs a quick test checklist, I can put one together.

10/7/2026,11:26:44AM



Here's a workflow you can repeat after each task. I'm assuming your project layout is a `game/` package (with `game_engine.py` and `color_button.py`) and `main.py` sitting next to it, since `main.py` imports `from game.game_engine import GameEngine`.

Every time, from the project root (the folder containing `main.py`):

```
# 1. Make sure pygame is installed (only needed once)
pip install pygame

# 2. Quick syntax check after pasting the new file
python -m py_compile game/game_engine.py

# 3. Run the game
python main.py
```

If `pip` or `python` isn't found, try `pip3` / `python3`. If you use a virtual environment, activate it first.

Optional but good for a lab: commit after each task, so you have a clean history and can roll back if something breaks:

```
git add game/game_engine.py
git commit -m "Task 1: fix sequence duplication bug"
```

(Use a matching message for each task. If you haven't run `git init` yet, do that once at the start.)

What to check by hand in the running game, per task:

- Task 1 (sequence bug):** Play a few rounds. Round 1 should flash 1 pad, round 2 flash 2, round 3 flash 3. Each round should add exactly one step.
- Task 2 (acceleration):** Get to a score of 8 to 10 and confirm the flashes are visibly faster than in round 1, and that they stop getting faster around score 12.
- Task 3 (sound):** Each pad should play a different pitch, both during playback and when you click. To test the no-audio fallback, you can temporarily run with a dummy driver: `SDL_AUDIODRIVER=dummy python main.py` (on Windows PowerShell: `$env:SDL_AUDIODRIVER="dummy"; python main.py`). The game should still run.
- Task 4 (timer):** Let the bar run out on round 1 and confirm "TIME'S UP!" appears. Then click a wrong pad and confirm "WRONG PATTERN! GAME OVER" appears. Press R after each to confirm restart works.

If something fails:

- `ModuleNotFoundError: No module named 'game'` means you're running from the wrong folder. `cd` to the folder that contains `main.py`. Also check that `game/` has an `__init__.py` file. Depending on your Python version it may not be strictly required, but adding an empty one is harmless.
- `ModuleNotFoundError: No module named 'pygame'` means pygame isn't installed in the Python you're running. Run `pip install pygame`, or `python -m pip install pygame` to be sure it matches your interpreter.
- A traceback after pasting a file usually means a copy/paste slip. Re-run step 2 (`py_compile`) and it will point at the line.

If your lab handout asks for specific commands (like a test runner or a particular branch workflow), follow those instead and I can help adapt the above.



Run these from your project root (the folder containing `main.py`).

One-time setup

```
# Tell git who you are (once per machine)
git config --global user.name "Your Name"
git config --global user.email "you@example.com"

# Start a repo (skip if you already did this or cloned one)
git init
git branch -M main
```

If you're pushing to GitHub, create an empty repository there first, then connect it:

```
git remote add origin https://github.com/YOUR-USERNAME/YOUR-REPO.git
```

(If you cloned the repo instead, `origin` is already set. Check with `git remote -v`.)

After each task

```
git status           # see what changed
git add game/game_engine.py  # stage the file you edited
git commit -m "Task 1: fix sequence duplication bug"
git push -u origin main  # first push only
```

For later pushes, `git push` alone is enough:

```
git add game/game_engine.py
git commit -m "Task 2: add dynamic playback acceleration"
git push
```

Notes

- **Staging everything:** `git add .` stages all changed files, but check `git status` first so you don't commit things like `__pycache__/_`. You can ignore those by creating a `.gitignore` file containing `__pycache__/_` and `*.pyc`.
- **Authentication:** GitHub no longer accepts your account password for pushes over HTTPS. When prompted, use a personal access token as the password (GitHub → Settings → Developer settings → Personal access tokens), or install the GitHub CLI and run `gh auth login`, or set up SSH and use the `git@github.com: USER/REPO.git` URL instead.
- **Push rejected ("fetch first" or "non-fast-forward"):** The remote has commits you don't have locally, for example a README created on GitHub. Run `git pull --rebase origin main`, then `git push` again.
- **Wrong commit message:** If you haven't pushed yet, `git commit --amend -m "new message"` fixes it.
- **Checking history:** `git log --oneline` shows your commits, which is a nice way to confirm you have one per task.

If your lab uses a different branch name (like `master`) or requires a specific workflow, swap `main` for that name.



and i do this for all tasks?

10/7/2026,11:52:36PM



Clarifying whether the process applies to every task. >

Yes, but only the "After each task" part repeats. The one-time setup (`git config`, `git init`, `git remote add`) is done once at the start.

For each task:

```
# 1. Paste the new game_engine.py, then check it runs
python main.py
```

```
# 2. Commit and push
git add game/game_engine.py
git commit -m "Task N: short description"
git push
```

The very first push needs `git push -u origin main`. After that, plain `git push` works.

Each task's file includes the previous tasks' changes, so the Task 4 file is the final, complete version. For one commit per task, commit Task 1's version first, then replace the file with Task 2's version and commit again, and so on. If you've already pasted the Task 4 version, just make one commit for it, or ask your instructor whether per-task commits are required.

10/7/2026,11:52:40PM